WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

神农微服务开发套件介绍



主讲人：眸望

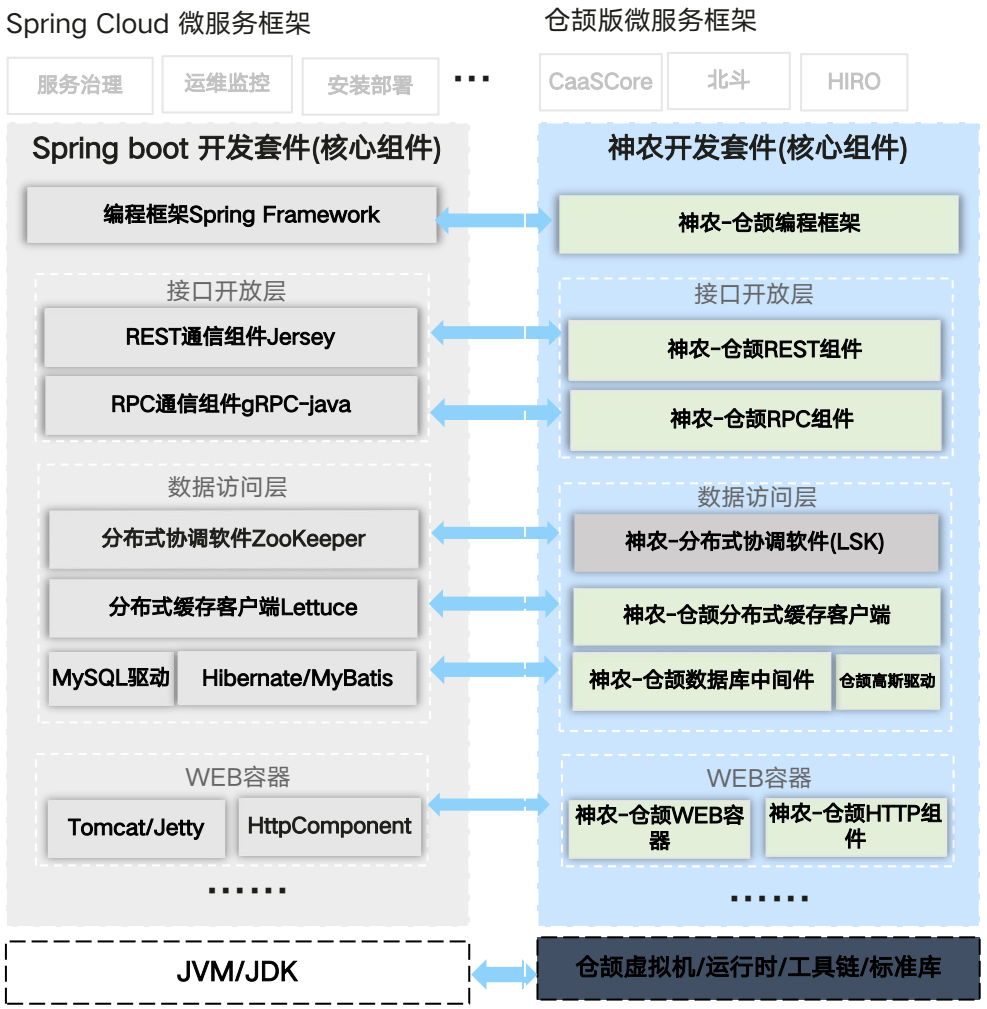
01 整体架构

02 开发框架

03 Rest&ORM

04 全链路异步

目标：仓颉微服务开发套件基础功能构筑，提供业界第二选择



目标1：打造仓颉版开发套件(神农)：完成编程框架等8+核心软件基础功能，逐步打造仓颉版开发套件，**具备类SpringBoot体系的能力**

1、**基于语言特性**完成基础功能构筑：充分利用仓颉语言多模式特点，发挥函数编程，宏处理等优势，优化面向对象编程下开发框架设计过于复杂和性能开销大的问题。

2、**基于声明式API**：从组件的易集成，易使用入手，提供一站式的开发套件对应的声明式编程API，对业务代码无侵入。

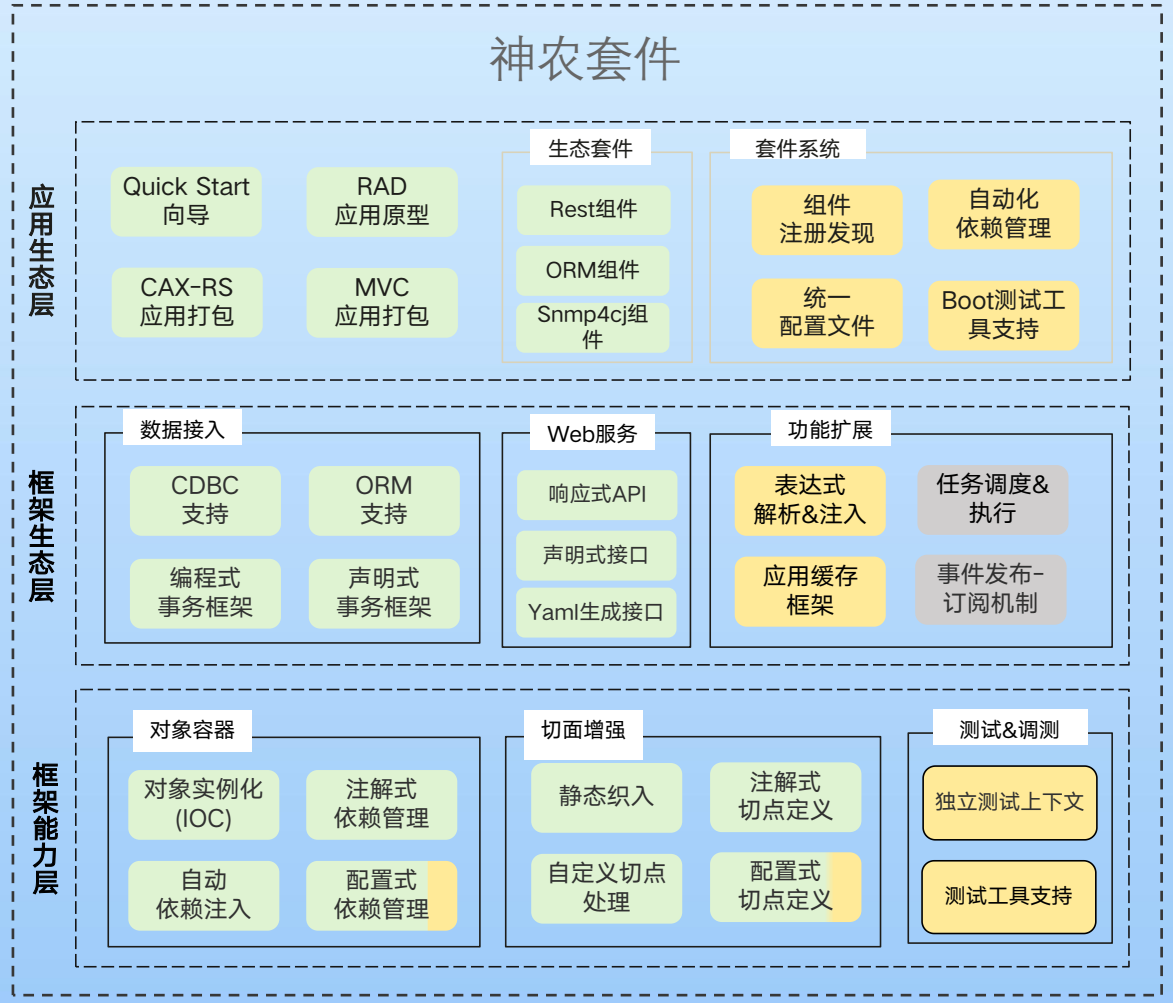
目标2：协同仓颉语言，打造全栈原生支持响应式编程、全异步无阻塞的下一代微服务应用开发技术栈。

1、**利用语言优势：**基于仓颉协程技术打造异步编程体系，充分利用语言的低开销，高性能的优势。

2、注重**端到端性能**：采用响应式和全异步方案，提高端到端性能。

仓颉 Cangjie

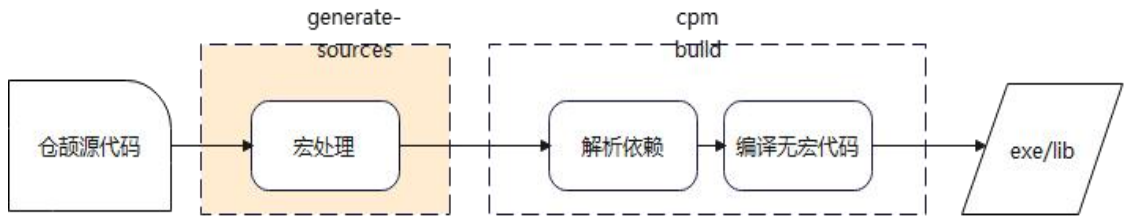
变化：在基础能力之上，增加了对测试、缓存、集成系统的支持。



	2023	2024
Ioc容器	✓	
aop框架	✓	增强
数据库事务	✓	
集成系统		✓
RestAPI	✓	增强
测试支持		✓
缓存API		✓
ORM	✓	增强
表达式		✓
自定义扩展		✓
应用打包		✓

开发框架关键特性：IOC&AOP

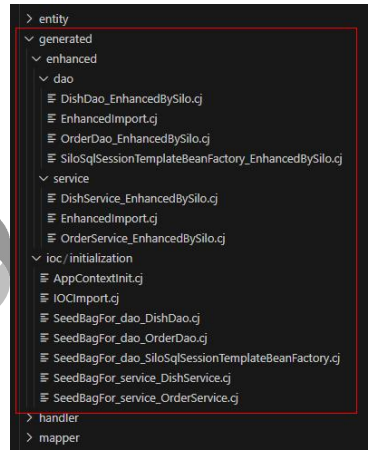
核心技术：基于AOT，而非反射




```
@Component
public open class DishService {
    let dishDao: DishDao

    @Inject["dish.dao.DishDao"]
    public init(dishDao: DishDao) {
        this.dishDao = dishDao
    }

    /**
     * 显示所有菜品
     */
    public func displayAllDishes(): String {
        let allDishInfoList: ArrayList<DishInfo> = dishDao.queryAllDishInfo()
        var allDish = StringBuilder()
        for (dishInfo in allDishInfoList) {
            allDish.append(dishInfo.toString() + "\n")
        }
        return allDish.toString()
    }
}
```




```
public func getAppContext(): SiloContext {
    let pluginCenter = PluginCenter()
    let storeHouse = AotSeedStore()
    let siloContext = SiloContext(storeHouse, pluginCenter)
    storeHouse.register(SeedBagFor_silo_aop_AopCenter(), pluginCenter, lazyInit: false)
      storeHouse.register(
        SeedBagFor_dish_dao_OrderDao(storeHouse.getSeedBag("sqlTemplate")),
        pluginCenter,
        lazyInit: false
    )
    storeHouse.register(SeedBagFor_dish_dao_DishDao(storeHouse.getSeedBag("sqlTemplate")), pluginCenter, lazyInit: false
    )
    storeHouse.register(
        SeedBagFor_dish_service_OrderService(storeHouse.getSeedBag("orderDao"), storeHouse.getSeedBag("dishDao")),
        pluginCenter,
        lazyInit: false
    )
    storeHouse.register(
        SeedBagFor_dish_service_DishService(storeHouse.getSeedBag("dishDao")),
        pluginCenter,
        lazyInit: false
    )
    return siloContext
}
```

IOC、AOP支持情况

特性	功能点	AOT支持情况
IOC容器	自动实例化	支持
	依赖注入（构造函数方式）	支持
	依赖注入（set方法方式）	支持
	Lazy、单例、原型	支持
	工厂模式	支持
	Config模式	支持
配置注入	自动反序列化模型	支持
	声明式配置传递	支持
	Field注入	支持
切面编程	前置处理（Before）	支持
	后置处理（After）	支持
	环绕处理（Around）	支持
	异常处理（AfterThrow）	支持
	自定义切面（or 切点）	支持
	自定义处理器（Advice）	支持

数据库事务支持情况

当前已支持事务属性	功能点	AOT支持情况
事务传播模式	DEMANDED	支持
	PREFERRED	支持
	OBLIGATORY	支持
	DEMANDS_NEW	支持
	UN_PREFERRED	支持
	NOT_AT_ALL	支持
	EMBEDDED	支持
隔离级别	Unspecified	支持
	ReadCommitted	支持
	ReadUncommitted	支持
	RepeatableRead	支持
	Snapshot	支持
	Serializable	支持
	Linearizable	支持
	Chaos	支持
访问模式	ReadOnly	支持
	ReadWrite	支持
其他	事务超时时间	支持
	异常回滚	支持
	异常不回滚	支持
	自定义异常处理类型	支持

采用代码生成+AOP切面方式实现事务框架

```
@Tx[rollbackWhen = [ TimeoutException, Exception ]]
public open func saveUsersWithError(users: Array<UserDTO>): Bool {
    userdao.saveUser(UserTranslater.tranlateDTO(users[0]))
    userdao.saveUserWithError(UserTranslater.tranlateDTO(users[1]))
    return true
}
```

直接生成事务增强代理类，并纳入AOP框架

```
public func afterThrowingTxsaveUsersWithError(joinPoint: JoinPoint): Unit {
    let rollbackWhenList: Array<String> = ["TimeoutException", "Exception"]
    let rollbackExceptList: Array<String> = []

    if (let Some(exception) <- joinPoint.getException()) {
        let exceptionStr = exception.toString()
        var exceptionClassName = "Exception"
        if (let Some(colonIndex) <- exceptionStr.indexOf(":")) {
            exceptionClassName = exceptionStr[0..colonIndex]
        }

        let needRollback = rollbackWhenList.contains(exceptionClassName) &&
            !rollbackExceptList.contains(exceptionClassName)
        if (needRollback) {
            manager.markShouldRollback()
        }
    }
}
```

Rest组件基础功能

特性	功能点	是否支持
RestServer请求分发	资源映射(@Rest)	支持
	资源映射(@GET)	支持
	资源映射(@POST)	支持
	资源映射(@PUT)	支持
	资源映射(@PATCH)	支持
	资源映射(@DELETE)	支持
	媒体类型(@Consumes)	支持
	路径参数（@PathParameter）	支持
	查询参数（@QueryParamter）	支持
	请求头参数（@HeaderParameter）	支持
	上下文（@Context）	
RestServer扩展配置	Cookie参数（@CookieParameter）	后续支持
	媒体类型(@Produces)	后续支持
RestClient	自动序列化/反序列化	支持
	请求参数配置（RestOption）	支持

RestServer按照CAX-RS规范进行实现，依托于仓颉元编程能力，实现了能够以类似标签的方式进行web服务的自动发现与注册的功能。

RestClient在仓颉Http基础之上进行了二次封装开发，提供了基于模型的Rest请求能力，尽可能贴近华为现有Rest开发实际使用方式，无论是全新开发还是切换开发都能够应对。

RestServer

> Rest组件实现了CAX-RS标准，提供了Restful风格报文与业务模型对象的自动映射已经URL到指定方法的路由

Rest组件的使用同样采用元编程模式，下面将对主要的宏进行样例说明。

@Rest

> @Rest宏使用在类上，表明该类是一个Rest资源。

使用样例

```
@Rest["/cutover/cangjie/device"]
public class DeviceServer {

}
```

@Res宏可以提供参数URL，该类下的所有方法都会以该URL作为前缀进行路由。

@POST、@GET、@PUT、@PATCH、@DELETE

> 以上标签使用在标记为@Rest类中的方法上，用以对应不同的HTTP请求类型和方法。映射方式参考CAX-RS

使用样例

RestClient

> Rest组件在仓颉HttpClient之上进行了封装，提供了能够发送Rest风格对象的RestClient。

使用样例

```
public class DeviceClient {
    private static let rest: Restful = getConfiguredRestInstance() //Rest客户端


    public static func createDevice(name: String, esn: String): RestfulResponse {
        println("[client] create device")

        let deviceDto: DeviceDto = DeviceDto(name, esn)
        let param =
            RestfulParameters()
            .addHttpContextHeader("Content-Type", "application/json")
            .setRawData(deviceDto.toJsonString())
```

仓颉 Cangjie

Rest组件支持自定义媒体类型&拦截器

支持Json格式的自定义媒体类型，对Json格式的数据进行资源实体映射与Bean校验处理。

支持自定义拦截器，实现自定义权限校验、熔断等功能的底层支持

使用示例：

Before

1 定义Bean模型

```
public class User {
    public var id: String = "0"

    public var userName: String = ""

    public var userAge: Int32 = 0
}
```

2 接口请求用到此模型

```
@Post["/user"]
@RequestBody
public func addUser(user: User): RestResponse {
    let regex = Regex("\\d{1,10}")
    let matcher: Matcher = regex.matcher(user.id)
    let matchData: Option<MatchData> = matcher.find()
    if (isOptionNone(matchData)) {
        return RestResponseImpl().badRequest("The user id is not valid".toUtf8Array())
    }
    if (user.userName.size<1) {
        return RestResponseImpl().badRequest("The user name's length is small than 1".toUtf8Array())
    } else if(user.userName.size>32) {
        return RestResponseImpl().badRequest("The user name's length is bigger than 32".toUtf8Array())
    }
    if (user.userAge<0) {
        return RestResponseImpl().badRequest("The user age is small than 1".toUtf8Array())
    } else if(user.userAge>100) {
        return RestResponseImpl().badRequest("The user age is bigger than 100".toUtf8Array())
    }

    let business: UserBusiness = SiloContext.siloAppContext.getSeed("userBusiness")
    let id = user.id
    try {…
    } catch (e: Exception) {…
    }
    return RestResponseImpl().ok("Save user success".toUtf8Array())
}
```

WARN：客户端数据直接进行到addUser方法内 ，如果不作校验则会直接存储到DB中，可能会引发未知的错误，所以需要添加校验语句。并且每个客户侧数据都需要做此校验处理。

After

1 定义Bean模型和校验规则

```
@AutoSerializable
public class User {
    @Property[name="id", pattern="\\d{1,10}"]
    public var id: String = "0"

    @Property[name="userName", minLength=1, maxLength=32]
    public var userName: String = ""

    @Property[name="userAge", min=0, max=100]
    public var userAge: Int32 = 0
}
```

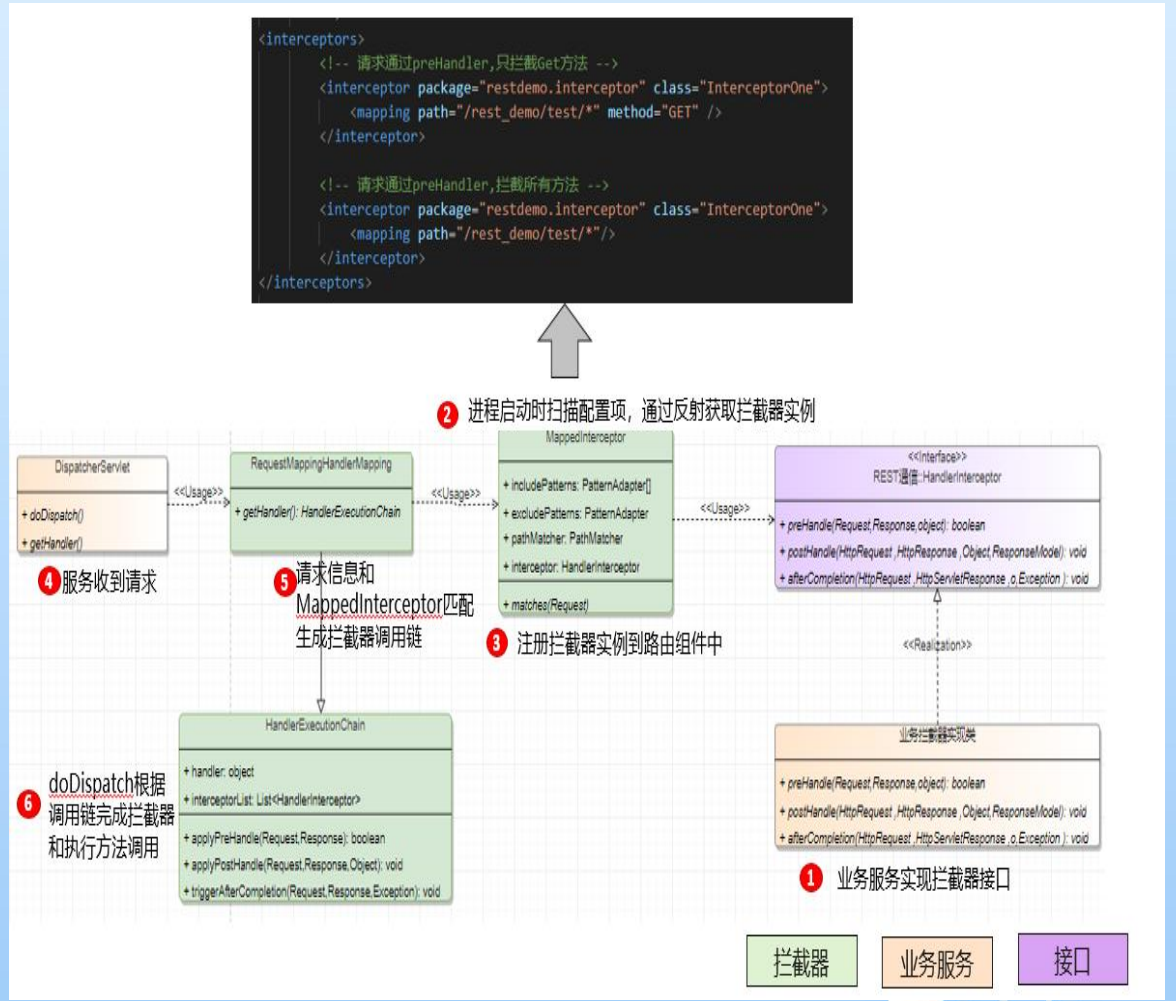
2 接口请求用到此模型，使用时无变化，无侵入

```
@Post["/user"]
@RequestBody
public func addUser(user: User): RestResponse {
    let business: UserBusiness = SiloContext.siloAppContext.getSeed("userBusiness")
    let id = user.id
    try {…
    } catch (e: Exception) {…
    }
    return RestResponseImpl().ok("Save user success".toUtf8Array())
}
```

INFO：用了校验后，校验由Bean校验模块在addUser方法前完成，校验不符合规则的用户数据。不需要在addUser里写复杂的校验代码

```
<interceptors>
    <!-- 请求通过preHandler,只拦截Get方法 -->
    <interceptor package="restdemo.interceptor" class="InterceptorOne">
        <mapping path="/rest_demo/test/*" method="GET" />
    </interceptor>

    <!-- 请求通过preHandler,拦截所有方法 -->
    <interceptor package="restdemo.interceptor" class="InterceptorOne">
        <mapping path="/rest_demo/test/*"/>
    </interceptor>
</interceptors>
```



仓颉 Cangjie

• ORM

基本标签支持情况

支持能力	Mybatis	Spirit
sql	支持	支持
Include标签	支持	支持
Namespace	支持	支持
@Id，@Entity，@Column，@Table	支持	支持
OneToMany、ManyToOne、OneToOne、ManyToMany、JoinColumns、JoinColumn、jointable	支持	支持
convert注解实现实体字段类型自定义转换	支持	支持
@GenerstedValue，GenerationType、SequenceGererator等注解实现主键生成免配置	支持	支持
基于FetchType注解实现延迟加载免配置	支持	不支持
基于StoredProcedureQuery注解实现存储过程免配置	支持	不支持
基于方法约定的免配置	支持	不支持

多数据源支持

背景：用户转型过程中数据隔离，不同的数据存在不同的数据库中，业务过程中涉及到多个数据源。

支持能力	mybatis	Spirit	特性说明
多数据源配置	支持	支持	在一个业务内配置多个数据源
多数据源DDL、DML等能力	支持	支持	多个数据源的数据定义以及数据操作能力
多数据源内事务管理	支持	支持	多个数据源场景下，每个数据源内的事务管理

使用样例

```
@Component[alias = "datasource", seedModuleName = "", seedPackageName = "std.database.sql"]
public open class DatasourceBean <: FactorySeed<Datasource> {
    public func getObject(): Datasource {
        return getDatasource()
    }

    func getDatasource() {
        let managerdDataSource = DriverManagerDatasource(
            [redacted]
        )
        return managerdDataSource.datasource
    }
}
```

❶配置多个数据源

```
@Component[alias = "sqlTemplate", seedModuleName = "", seedPackageName = "silo_spirit.silo.spirit"
public open class SiloSqlSessionFactoryBean <: FactorySeed<SiloSpiritSqlSessionTemplate> {
    private var siloSqlSessionFactory = Option<SiloSpiritSqlSessionTemplate>.None

    let datasource: Datasource
    @Inject["datasource"]
    public init(datasource: Datasource) {
        this.datasource = datasource
    }

    public func getObject(): SiloSpiritSqlSessionTemplate {
        let newSiloSqlSessionFactory = DaoFactoryBuilder(ConfigType.Mybatis).build(
            "src/test[redacted]", datasource, DataSourceRegister(datas
        newSiloSqlSessionFactory.getConfiguration().transactionFactory = ManagedTransactionFactory
        return SiloSpiritSqlSessionTemplate(newSiloSqlSessionFactory)
    }
```

❷配置多个SessionProxy，每个绑定指定的数据源

```
@Component[alias ="siloManager", seedModuleName ="", seedPac
public open class SiloTransactionManagerFactoryBean <: Facto

    let datasource: Datasource
    @Inject["datasource"]
    public init(datasource: Datasource) {
        this.datasource = datasource
    }

    public func getObject(): SiloTransactionManager {
        return SiloTransactionManagerImpl(datasource)
    }
```

❸配置多个事务管理器，每个绑定指定的数据源。使用事务时绑定需要的事务管理器。

仓颉 Cangjie

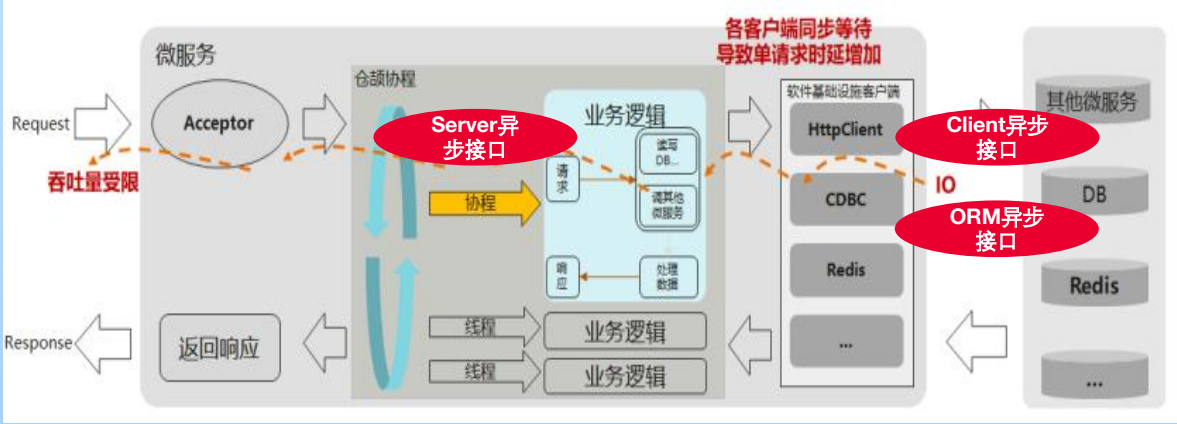
全链路异步

响应式编程简介

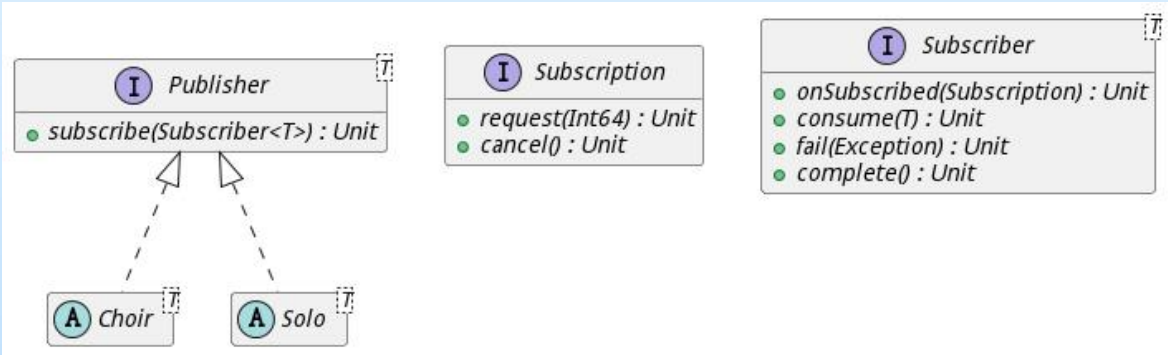
响应式编程是一种异步的编程范式，它可以帮助开发人员更容易地处理异步操作、并发问题和高性能需求。

响应式异步非阻塞式编程相比传统同步阻塞式编程优劣势

方式	及时性	弹性	资源利用	错误处理
传统同步阻塞	中等	低	低	简单
响应式异步非阻塞	高	高	高	复杂



仓颉响应式编程框架和Rest实现



- Publisher用以产生数据，并将所产生的数据推给订阅到其的Subscriber处理。
- Subscriber用以处理数据。
- Subscription表示一份订阅关系，同时向Subscriber提供Publisher的操作界面。

```
@Get["/get/choir"]
public func getChoir(): Choir<String> {
    return Choir.create(
        {
            emitter: Emitter<String> =>
            let begin = DateTime.now()
            var i = 0;
            while (i < 100) {
                emitter.emit("event " + i.toString())
                sleep(Duration.second)
                i++
            }
            emitter.complete()

            let cost = DateTime.now() - begin
            println("Create cost time ${cost.toMilliseconds()}")
        }
    )
}
```

```
context.responseBuilder.header("Content-Type", "text/event-stream")
context.responseBuilder.header("Cache-Control", "no-cache")
context.responseBuilder.header("Connection", "keep-alive")
let writer = HttpResponseWriter(context)
handlerResult.subscribe(
consume: {
    data => writer.write(serialize(data).toUtf8Array())
},
fail: {
    ex: Exception => exList.append(ex.message)
},
complete: {
    => println("The data in Flowable has been consumed.")
}
)
```

- Rest服务端作为Publisher，支持发布响应式Choir和Solo数据
- Rest客户端作为Subscriber处理响应式数据

仓颉 Cangjie

WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

关注“仓颉编程语言”公众号，
获取最新官方资讯!

